SQL Notes - Every Line Explained

Tables, Constraints, ALTER, INSERT, SELECT, Casting + Practice Tasks

How to read: green lines starting with `--` are comments (ignored by SQL, written for humans). Dark lines are the actual SQL. Run the script top to bottom in PostgreSQL.

1. Resetting the Database (DROP TABLE)

Before building anything we clear old tables so the script can be re-run safely any number of times.

```
-- DROP TABLE removes a table AND all its data permanently.
-- IF EXISTS = do not throw an error if a table is missing (makes the script re-runnable).
-- We list 3 tables separated by commas, so one command deletes all three.
-- CASCADE = also automatically drop anything that depends on these tables
--           (e.g. foreign keys in other tables pointing at them). Without it,
--           PostgreSQL refuses to drop a table that another table references.
-- The semicolon (;) marks the END of an SQL statement.
DROP TABLE IF EXISTS orders, products, customers CASCADE;
```

2. Creating Tables (CREATE TABLE, Data Types, Constraints)

CREATE TABLE defines the structure: column names, data types and rules (constraints). Parent tables (customers, products) are created first; the child table (orders) comes last because it references them.

```
-- CREATE TABLE <name>( ... ) builds a new empty table.
-- Inside the brackets each line is:  column_name  DATA_TYPE  [constraints],
CREATE TABLE customers(
    -- SERIAL = auto-incrementing integer (1, 2, 3 ...). You never type it yourself.
    -- PRIMARY KEY = unique + NOT NULL; it identifies each row exactly once.
    customer_id  SERIAL PRIMARY KEY,
    -- VARCHAR(50) = text of up to 50 characters.
    -- NOT NULL = this column can never be empty; every customer must have a name.
    name         VARCHAR(50) NOT NULL,
    -- VARCHAR(30) with no NOT NULL = optional; NULL (unknown) is allowed.
    city         VARCHAR(30),
    -- DATE stores only a calendar date in YYYY-MM-DD format (no time).
    signup_date  DATE
);   -- closing bracket ends the column list, semicolon ends the statement.

-- Second table: the catalogue of things we sell.
CREATE TABLE products(
    -- Same auto-numbered primary key idea as above.
    product_id      SERIAL PRIMARY KEY,
    -- UNIQUE = no two products may share the same name (duplicates are rejected).
    product_name    VARCHAR(50) NOT NULL UNIQUE,
    -- Category such as Electronics / Furniture / Books (optional).
    category        VARCHAR(30),
    -- NUMERIC(10, 2) = exact decimal: up to 10 digits in total, 2 after the decimal point.
    --   Use NUMERIC (not FLOAT) for money so there are no rounding errors.
    -- CHECK (price > 0) = a rule evaluated on every insert/update; a price of 0 or
    --   a negative price is rejected with an error.
    price           NUMERIC(10, 2) CHECK (price > 0)
);

-- Third table: each row is one order made by one customer for one product.
CREATE TABLE orders(
    order_id      SERIAL PRIMARY KEY,
    -- INT = whole number.
    -- REFERENCES customers(customer_id) = FOREIGN KEY. The value here must already
    --   exist in customers.customer_id. This links the two tables and prevents
    --   orders for customers that do not exist.
    customer_id   INT REFERENCES customers(customer_id),
```

```
    -- Same idea: links each order to a row in products.
    product_id    INT REFERENCES products(product_id),
    -- DEFAULT 1 = if you do not supply a quantity, 1 is stored automatically.
    -- CHECK (quantity > 0) = quantity must be a positive number.
    quantity      INT DEFAULT 1 CHECK (quantity > 0),
    -- NOT NULL = an order must always have a date.
    order_date    DATE NOT NULL,
    -- Free text status. The "--" comment on the original line lists the values we use:
    -- 'Delivered', 'Cancelled', 'Returned'.
    status        VARCHAR(20)
);
```

3. ALTER TABLE (change a table that already exists)

ALTER modifies the structure of an existing table without deleting its data. These lines are commented out in the notes (a leading -- stops SQL from running them). Remove the -- to try one; run them in order.

```
-- ALTER : change a table that already exists
-- (This line is itself a comment: any text after -- on a line is ignored by SQL.
--  Comments are for humans; they document the code.)

-- ADD COLUMN: adds a brand-new column called email (text up to 100 chars) at the end.
-- Existing rows get NULL in the new column.
-- ALTER TABLE customers ADD COLUMN email VARCHAR(100);

-- RENAME COLUMN ... TO ...: changes only the column's name; data stays untouched.
-- ALTER TABLE customers RENAME COLUMN email TO email_id;

-- ALTER COLUMN ... TYPE: changes the data type / size, here from 100 to 150 characters.
-- ALTER TABLE customers ALTER COLUMN email_id TYPE VARCHAR(150);

-- ADD CONSTRAINT: attaches a named rule to an existing table.
-- uq_email is the name we give the rule; UNIQUE (email_id) means no duplicate emails.
-- NOTE: the original line is missing the ending semicolon; always finish with ;
-- ALTER TABLE customers ADD CONSTRAINT uq_email UNIQUE (email_id);

-- DROP COLUMN: removes the column and all its data. A constraint that depends only
-- on that column (like uq_email) is removed along with it.
-- ALTER TABLE customers DROP COLUMN email_id;
```

4. Schemas (organising tables into folders)

A schema is a named folder inside a database used to group related tables. The default schema is called public. These lines are also commented out.

```
-- CREATE SCHEMA creates a new schema named reports.
-- IF NOT EXISTS = do nothing (no error) if it already exists.
-- CREATE SCHEMA IF NOT EXISTS reports;

-- schema_name.table_name (reports.demo) puts the table inside the reports schema.
-- IF NOT EXISTS on a table works the same way as on a schema.
-- The table has just one column, id, of type INT.
-- CREATE TABLE IF NOT EXISTS reports.demo  (id INT);

-- DROP SCHEMA removes the schema; CASCADE also removes every table inside it.
-- Without CASCADE the command fails if the schema is not empty.
-- DROP SCHEMA reports CASCADE;
```

5. DROP vs TRUNCATE vs DELETE (key concept)

These three commands look similar but remove different things. This is a very common interview question.

```
-- DROP TABLE deletes the whole table.
```

```
--   Structure + data + constraints are all gone. The table no longer exists.
--   Example: DROP TABLE orders;
--
-- TRUNCATE keeps the table but empties all rows instantly.
--   The structure stays; only the data is removed. It is very fast, takes NO WHERE
--   condition (all rows go) and can reset the SERIAL counter with RESTART IDENTITY.
--   Example: TRUNCATE TABLE orders RESTART IDENTITY;
--
-- DELETE removes chosen rows.
--   Used with a WHERE condition to remove only the rows you pick; slower than
--   TRUNCATE on big tables but precise. Without WHERE it deletes every row.
--   Example: DELETE FROM orders WHERE status = 'Cancelled';
--
-- Quick memory trick:
--   DROP = remove the table itself | TRUNCATE = empty it | DELETE = remove some rows
```

6. Inserting Data (INSERT INTO ... VALUES)

INSERT adds new rows. We name the columns in brackets, then give one bracket of values per row. We skip the SERIAL id columns because PostgreSQL generates them automatically.

```
-- INSERT INTO <table> (<columns>) VALUES (<values>), (<values>), ...;
-- Listing the columns means the values must follow the SAME order.
-- customer_id is left out on purpose - SERIAL fills it with 1, 2, 3 ...
INSERT INTO customers (name, city, signup_date) VALUES
-- Text and dates are wrapped in single quotes. Dates use the YYYY-MM-DD format.
-- Each row is one set of brackets; rows are separated by commas.
('Aarav Mehta',    'Gurugram',  '2025-01-05'),
('Priya Sharma',   'Delhi',     '2025-01-18'),
('Rohit Verma',    'Noida',     '2025-02-02'),
('Sneha Kapoor',   'Gurugram',  '2025-02-20'),
('Vikram Singh',   'Delhi',     '2025-03-11'),
-- Messy on purpose: city has extra spaces and lower-case (' mumbai ').
-- Later we practise cleaning it with TRIM() and INITCAP().
('Ananya Gupta',   ' mumbai ',  '2025-03-25'),
-- NULL means "no value / unknown". Note NULL has no quotes. It is allowed because
-- the city column does not have NOT NULL.
('Karan Malhotra', NULL,        '2025-04-08'),
-- The LAST row ends with a semicolon, not a comma.
('Isha Reddy',     'Bengaluru', '2025-04-22');

-- Now the products. product_id again auto-generates (1 to 6).
-- Numbers (prices) are written WITHOUT quotes.
INSERT INTO products (product_name, category, price) VALUES
('Laptop',       'Electronics', 55000),
('Headphones',   'Electronics',  2500),
('Office Chair', 'Furniture',    8000),
('Desk Lamp',    'Furniture',    1200),
('Python Book',  'Books',         650),
('SQL Book',     'Books',         550);

-- Now the orders. The numbers in customer_id / product_id are the ids created above.
-- Because of the foreign keys, an id that does not exist (e.g. customer 99) would FAIL.
INSERT INTO orders (customer_id, product_id, quantity , order_date, status) VALUES
-- Customer 4 (Sneha) bought product 5 (Python Book), 10 copies, on 19 Jan 2026.
(4 , 5, 10 , '2026-01-19' , 'Delivered' ),
-- Customer 1 (Aarav) bought 2 Headphones.
(1, 2, 2, '2025-02-14', 'Delivered'),
-- Customer 2 (Priya) bought 3 Python Books.
(2, 5, 3, '2025-01-25', 'Delivered'),
-- Status can be Cancelled: the order was placed but not completed.
(2, 3, 1, '2025-03-02', 'Cancelled'),
(3, 2, 1, '2025-02-10', 'Delivered'),
(3, 6, 2, '2025-03-15', 'Delivered'),
```

```
(4, 4, 2, '2025-03-01', 'Delivered'),
-- Status can be Returned: the customer sent a Laptop back.
(4, 1, 1, '2025-04-05', 'Returned'),
(5, 3, 2, '2025-03-20', 'Delivered'),
(5, 5, 1, '2025-04-12', 'Delivered'),
(6, 2, 1, '2025-04-02', 'Delivered'),
(6, 6, 4, '2025-05-06', 'Delivered'),
(1, 4, 1, '2025-05-10', 'Delivered'),
(8, 1, 1, '2025-05-18', 'Delivered'),
(2, 2, 1, '2025-05-21', 'Delivered');
```

7. Reading Data (SELECT)

SELECT is the most used SQL command. It reads data and never changes the table.

```
-- reading data
-- SELECT * FROM <table>; shows every column and every row.
-- * is a wildcard that means "all columns".
-- Good for quick checks; avoid * in real projects with many columns (slow, noisy).
SELECT * FROM orders;

-- Choose only the columns you need, separated by commas.
-- Order of columns in the result follows the order you write them.
SELECT  order_date, status FROM orders;

-- DISTINCT removes duplicate values from the result, so every city appears once.
-- NULL (Karan's missing city) is treated as a value too and shows as one blank row.
-- ' mumbai ' is its own value because of the extra spaces.
SELECT DISTINCT city FROM customers;
```

8. Calculated Columns and Aliases (AS)

SQL can do maths inside a SELECT. AS gives the calculated column a readable name (alias). The notes show three versions of the same GST query; the first two are commented out and the third is the final one.

```
-- VERSION 1 (commented out): basic calculation, shows raw long decimals.
-- SELECT
--     product_name,                        -- plain column from the table
--     price ,                              -- plain column from the table
--     price * 0.18  AS  gst ,              -- * multiplies; 0.18 = 18% GST; AS gst = alias
--     price * 1.18 AS price_with_gst       -- price + 18% in one step (1 + 0.18 = 1.18)
-- from products;                          -- FROM names the table (SQL keywords are case-insensitive)

-- VERSION 2 (commented out): ROUND the result with the :: cast operator.
-- (price * 0.18)::NUMERIC(10,2)  -> the brackets calculate first, then :: converts the
--   result to NUMERIC with 2 decimal places. :: is PostgreSQL's short cast syntax.
-- SELECT
--     product_name,
--     price ,
--     (price * 0.18)::NUMERIC(10,2)  AS  gst ,
--     (price * 1.18)::NUMERIC(10,2) AS price_with_gst
-- from products;

-- VERSION 3 (the one that runs): mixes both cast styles side by side.
SELECT
    -- Plain columns copied from the products table.
    product_name,
    price ,
    -- Short cast: calculate 18% GST, then force 2 decimal places. Alias = gst.
    (price * 0.18)::NUMERIC(10,2)  AS  gst ,
    -- CAST(value AS type) is the standard-SQL way of doing the same conversion;
    -- it works in almost every database, while :: works only in PostgreSQL.
    -- Result: price including 18% GST, rounded to 2 decimals, named price_with_gst.
```

```
    CAST(price * 1.18 AS NUMERIC(10,2)) AS price_with_gst
-- FROM tells SQL which table to read the rows from.
from products;
```

WebiGeeks

Practice Tasks (different tables, same topics)

For these tasks use a NEW set of tables for an institute database. Write every query yourself first, then run it and compare the output with what you expect. Solutions are intentionally not included.

Tables to use

```
students(student_id, full_name, city, join_date)
courses(course_id, course_name, category, fee)
enrollments(enrollment_id, student_id, course_id, seats_booked, enroll_date, status)

-- status values: 'Active', 'Dropped', 'Completed'
-- fee is money, join_date / enroll_date are dates
```

A. DROP, CREATE TABLE and Constraints

Task 1. Write a script that safely drops students, courses and enrollments (re-runnable, and works even if they do not exist or reference each other).

Task 2. Create students with: student_id as an auto-increment primary key, full_name (up to 60 chars, required), city (up to 30 chars, optional) and join_date.

Task 3. Create courses with: course_id (auto primary key), course_name (up to 50 chars, required and no duplicates), category (up to 30 chars) and fee that holds up to 8 digits with 2 decimals and must be greater than 0.

Task 4. Create enrollments with: an auto primary key, student_id and course_id as foreign keys to the other tables, seats_booked that defaults to 1 and must be greater than 0, enroll_date (required) and status (up to 15 chars).

Task 5. Explain in a comment why enrollments must be created after students and courses.

B. ALTER TABLE

Task 6. Add a column phone (up to 15 chars) to students.

Task 7. Rename phone to mobile_no.

Task 8. Change the type of mobile_no so it allows up to 20 characters.

Task 9. Add a rule named uq_mobile so no two students can share a mobile number.

Task 10. Remove the mobile_no column. What happens to uq_mobile? Write your answer as a comment.

C. Schemas

Task 11. Create a schema named archive, only if it does not exist already.

Task 12. Inside it, create a table old_students (id INT, full_name VARCHAR(60)), only if it does not exist.

Task 13. Remove the schema together with everything inside it.

D. DROP vs TRUNCATE vs DELETE

Task 14. Write the command to remove only the enrollments whose status is 'Dropped'.

Task 15. Write the command to empty the whole enrollments table and restart its id counter, keeping the table structure.

Task 16. Write the command that removes the courses table completely.

Task 17. In three comment lines, state one difference between each pair: DROP and TRUNCATE, TRUNCATE and DELETE, DELETE and DROP.

WebiGeeks

E. INSERT

Task 18. Insert 8 students with different cities. Make one city NULL and one city badly formatted with extra spaces and lower-case letters.

Task 19. Insert 6 courses across at least 3 categories (for example Programming, Analytics, Design) with different fees.

Task 20. Insert 14 enrollments that use a mix of the statuses Active, Dropped and Completed, with dates across several months. Leave seats_booked out in at least two rows so the default is used.

Task 21. Try inserting a course with fee = 0 and an enrollment for student_id 999. Note the errors you get and explain each one in a comment.

F. SELECT, DISTINCT, Calculations and Casting

Task 22. Show every column and every row of students.

Task 23. Show only full_name and join_date from students.

Task 24. List the distinct categories in courses.

Task 25. List the distinct combinations of student_id and status in enrollments.

Task 26. Show course_name, fee and a column fee_with_gst that adds 18% GST to the fee, using the :: cast to keep 2 decimals.

Task 27. Repeat the previous task using CAST(... AS ...) instead of ::, and give the aliases meaningful names.

Task 28. Show course_name, fee, a 10% early-bird discount amount, and the final fee after the discount, all rounded to 2 decimals.

Task 29. Show enrollment_id, seats_booked, and a column total_fee_for_booking that is seats_booked multiplied by 1500, as a NUMERIC with 2 decimals.

Task 30. Show join_date from students cast to TEXT, and the fee from courses cast to INT.

G. Mixed Challenge

Task 31. Write a full script from scratch for a library database (books, members, loans) that drops old tables, creates the three tables with primary keys, foreign keys, a UNIQUE rule, a CHECK rule and a DEFAULT, inserts at least 5 rows in each, and ends with three different SELECT queries (one with DISTINCT, one with a calculated column, one with CAST).

Task 32. Add comments on every line explaining what it does, as you would when teaching a beginner.

WebiGeeks